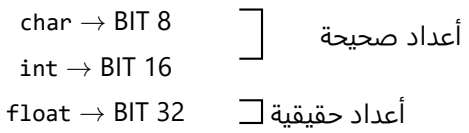
العوامل والطباعة على الشاشة

الدرس 4 — 1IOT، SST-ICT-01 — العوامل الحسابية، باقي القسمة، الإسناد، printf

21/09/2026

1 مراجعة على السبورة: أنواع البيانات الثلاثة

قبل تقديم العوامل، تذكير مصور قصير بما رأيناه في الدرس الماضي، كما كُتب على **السبورة** (**lavagna**):



شكل 1: كما كُتب على السبورة: char وint مجموعان كـ"أعداد صحيحة"، وfloat وحده كنوع "للأعداد الحقيقية" (الأعداد بفاصلة عشرية).

2 ما هي العوامل

تعريف

العامل (**operatore**) هو **الرمز** الذي يوضع بين **معاملين** (**operandi**، عناصر) أو أكثر **لدمجها** والحصول على **نتيجة**. وبحسب الرمز المستخدم تُنفَّذ **عملية** **مختلفة**.

مثال

في الرياضيات $5 + 2 = 7$: آخذ العنصرين 5 و2 وأدمجهما **بعامل** الجمع فأحصل على 7. وإذا دمجتهما **بالضرب** أحصل على 10: المعاملان نفسهما، عامل مختلف، **نتيجة** **مختلفة**.

3 العوامل الحسابية

تعريف

العوامل **الحسابية** هي التي تعلمناها في المدرسة الابتدائية: + (**الجمع**، **الطرح**، **somma** **sottrazio** النجمة)، / (**القسمة**، **divisione**، ويسمى **slash**). **moltiplicazione**، **الضرب**

ملاحظة

في المعلوماتية تُكتب **القسمة** **بالشرطة** **المائلة** (**slash**، /)، وليس **بالنقطتين** أو رمز القسمة المستخدم في الرياضيات.

3.1 عامل باقي القسمة

تعريف

الرمز % (يشبه **النسبة المئوية**، لكنه **ليس كذلك**) يسمى **عامل باقي القسمة** (**operatore modulo**): يعيد **باقي قسمة عددين صحيحين**.

مثال

كم يساوي $5 \div 2$ بالأعداد الصحيحة؟ نتيجة القسمة 2 و**الباقي** **1**. إذن $5 \bmod 2 = 1$.
كم يساوي $11 \bmod 3$؟ $11 \div 3 = 3$ (لأن $3 \times 3 = 9$)، و$11 - 9 = 2$: إذن $11 \bmod 3 = 2$.
كم يساوي $100 \bmod 30$؟ $100 \div 30 = 3$ (لأن $3 \times 30 = 90$)، و$100 - 90 = 10$: إذن $100 \bmod 30 = 10$.
أمثلة أخرى على السبورة: $16 \bmod 5 = 1$، $40 \bmod 9 = 4$.

تنبيه

يعطي عامل الباقي **الباقي فقط**، وليس **نتيجة القسمة**. لقيمة القسمة نحتاج إلى **slash** (/). فهم هذا العامل جيدًا **أساسي**: فسيُستخدم كثيرًا لاحقًا في البرمجة.

4 عامل الإسناد

تعريف

الرمز = في البرمجة هو **عامل الإسناد** (**operatore di assegnazione**): **ينسخ** القيمة الموجودة على **يمينه** إلى **المتغير** الموجود على **يساره**.



شكل 2: مخطط السبورة لعامل الإسناد: القيمة في الصندوق الأيمن تُنسخ إلى الأيسر – والسهم يتجه دائمًا من اليمين إلى اليسار.

مثال

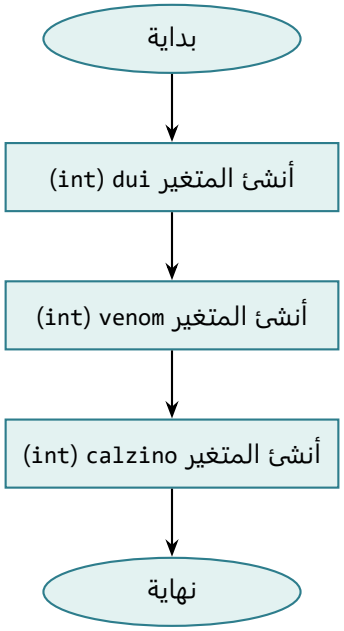
لننشئ ثلاثة **متغيرات** من نوع `int`، **بأسماء** **نختارها** **بحرية** (مثل `dui` و`venom` و`calzino` – لا يجب أن يرتبط الاسم بما يمثله، لكن **الاسم** **المعبّر** يساعد على إعادة قراءة الكود لاحقًا):

```
int dui;
int venom;
int calzino;
```

لا تنسَ **الفاصلة** **المنقوطة** (**punto e virgola**) في نهاية كل سطر: فهي تنهي الأمر.
ثم نسند القيم:

```
dui = 7;
venom = 10;
calzino = dui - venom;
```

هنا `dui` يحتوي 7، و`venom` يحتوي 10، و`calzino` يتلقى **نتيجة** `dui - venom`، أي -3: **ينفذ** الحاسوب **العملية** وينسخ القيمة الناتجة إلى المتغير الذي على اليسار.



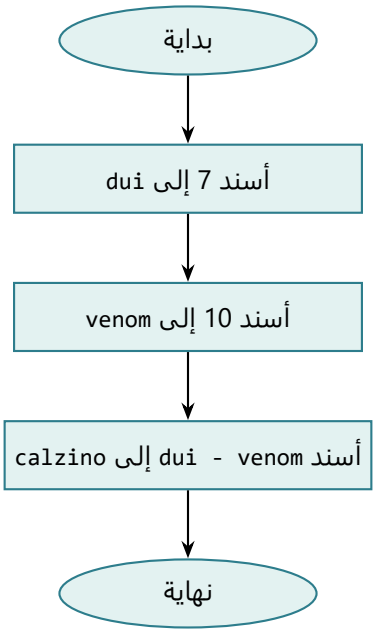
شكل 3: مخطط انسيابي (**diagramma di flusso**): تعريف المتغيرات الثلاثة من نوع `int` بالتسلسل.

ملاحظة

انتبه إلى **الفراغ** بين العامل والعدد: `dui = 7;` يعمل تمامًا مثل `dui=7;` (بدون فراغات) – فالمترجم (**compilatore**) **لا** **يفرق** بينهما – لكن الصيغة بالفراغات أسهل قراءة، فهي المفضلة.

تنبيه

الرمز = في البرمجة **ليس** رمز **المساواة** في الرياضيات. في الرياضيات "$x = 3$" يربط بين x و3، ونبحث عن قيمة x التي تجعل العبارة صحيحة. أما في البرمجة فإن = **ينسخ دائمًا** **من** **اليمين** **إلى** **اليسار**: كتابة `a = 5` لا معنى لها، لأنه لا يوجد **متغير** على اليسار (العدد مثل 5 لا يمكنه "**احتواء**" قيمة أخرى) – وهذا يسبب غالبًا **خطأ** **ترجمة** (**errore di compilazione**).



شكل 4: مخطط انسيابي: عمليات الإسناد الثلاث منفذة بالتسلسل، بالترتيب المكتوب في الكود.

5 قواعد أسماء المتغيرات

تعريف

يجب أن يلتزم اسم **المتغير** بهذه القواعد:

1. هو سلسلة **أبجدية** **رقمية**: يمكن أن يحتوي **حروفًا** **وأرقامًا**.
2. لا يمكن أن **يبدأ** **برقم** (يمكنك استخدام الأرقام، لكن ليس كأول حرف).
3. **لا** **يمكن** **أن** **يحتوي** **رموزًا** **خاصة**، باستثناء **underscore** (_، **trattino basso**) – وهو الاستثناء الوحيد المسموح.
4. هو **sensitive** **case** (يميز بين الحروف الكبيرة والصغيرة): `pippo` و`Pippo` متغيران **مختلفان**.
5. **لا يمكن أن يطابق كلمة محجوزة** في اللغة (مثل `int`، `char`، `return`، `for`): هذه كلمات تستخدمها لغة C لنفسها، ولا يمكن إعادة استخدامها كاسم متغير.

مثال

أمثلة على السبورة، طُلب فيها من القسم أن يقرر هل كل اسم صالح أم لا:

- `MICHELE` – **صالح**.
- `pippo11` – **صالح** (حروف+رقم، **لا يبدأ برقم**).
- `12LUIGI` – **غير صالح**: **يبدأ برقم**.
- `MAURO_BORTOLANI` – **صالح**: **underscore** هو الرمز الخاص الوحيد المسموح.
- `MAURO BORTOLANI` (مع **فراغ**) – **غير صالح**: الفراغ يقطع السلسلة، "فيكسر" الاسم إلى عنصرين منفصلين بالنسبة إلى اللغة.
- `a,b-5` – **غير** صالح: **الفاصلة** والشرطة **رموز خاصة** غير مسموحة.

مثال

Case sensitive عمليًا: `pippo` و`Pippo` و`PippoO` هي **ثلاثة متغيرات مختلفة** في نظر **المترجم**، حتى لو بدت للقارئ البشري "**الشيء نفسه تقريبًا**".

تنبيه

يمكن بدء الاسم بـ**underscore**، لكن المتغيرات التي تبدأ هكذا في لغة C لها **معنى تقني** خاص (متغيرات نظام/توجيه) لن نستخدمه في هذه الدورة – الأفضل تجنبه لعدم إثارة الالتباس.

مثال

تمرين موجَّه في القسم – لنعرّف ثلاثة **متغيرات** ونجرب استخدامها في حساب بعامل الباقي:

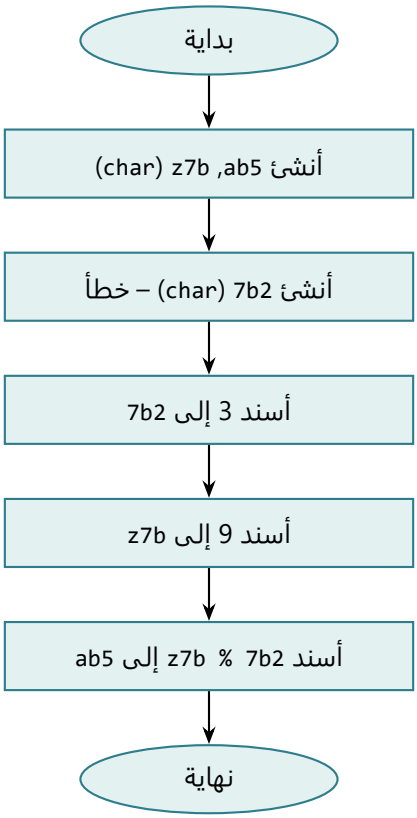
```
char ab5;
char z7b;
char 7b2;       // خطأ: الاسم يبدأ برقم!

7b2 = 3;
z7b = 9;
ab5 = z7b % 7b2;
```

الاسم `7b2` يخالف **القاعدة 2** (لا يمكن أن يبدأ برقم): **لا يُترجم** البرنامج. التصحيح بحذف **الرقم الأول**:

```
char ab5;
char z7b;
char b2;        // صحيح: يبدأ الآن بحرف

b2 = 3;
z7b = 9;
ab5 = z7b % b2;    // القيمة 9 % 3 = 0 يتلقى ab5
```



شكل 5: مخطط انسيابي للنسخة الخاطئة: يتوقف التسلسل عند الترجمة بسبب الاسم غير الصالح 7b2.

6 طباعة نتيجة على الشاشة: printf

حتى الآن تنفذ الآلة الحسابات لكنها لا تعرضها لنا: نحتاج إلى طريقة **لإظهار** **نتيجة** عملياتنا على **الشاشة** باستخدام الأمر **printf**.

تعريف

الأمر **printf** (كله بحروف صغيرة) **يطبع** (**stampa**) نصًا على الشاشة. يُكتب هكذا:

```
printf("testo");
```

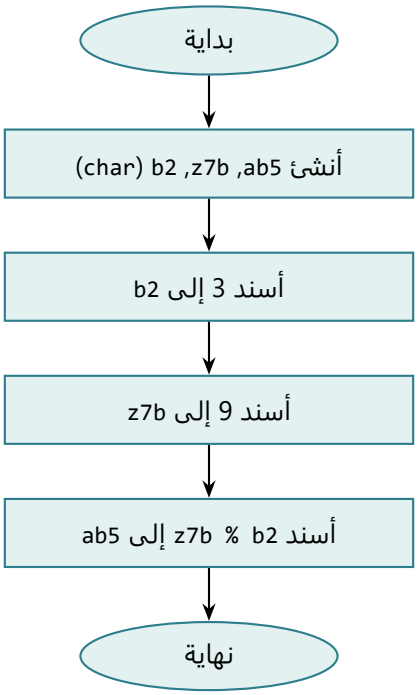
يوضع النص بين **علامتي** **اقتباس** **مزدوجتين** (**doppi apici**، ")، داخل القوسين.

مثال

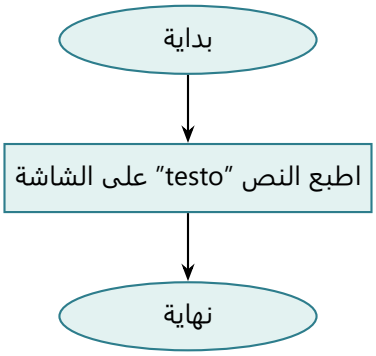
لطباعة **محتوى** **متغير** داخل النص نستخدم **محدد** **تنسيق** (**specificatore di formato**): الرمز %d (بالمئة-دي)، ويوضع في المكان الذي نريد أن **تظهر** **فيه** **القيمة**.

```
printf("%d", dui);
```

إذا كانت قيمة المتغير dui هي -3، فإن هذا الأمر **يطبع** **على** **الشاشة**: -3.



شكل 6: مخطط انسيابي للنسخة الصحيحة: يتلقى ab5 في النهاية القيمة $9 \bmod 3 = 0$.



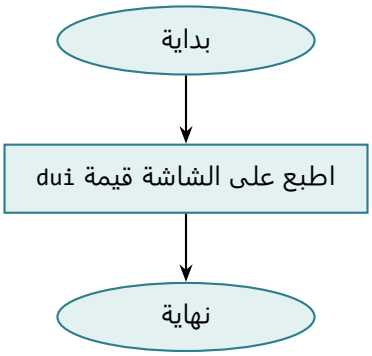
شكل 7: مخطط انسيابي: فعل واحد فقط، طباعة النص على الشاشة.

ملاحظة

الرمز % هنا لا علاقة له **بالنسبة المئوية الرياضية**: إنه يبدأ **محدد تنسيق**، أي **تعليمة خاصة** تقول "ضع في هذا المكان قيمة متغير"، وليس **حرفًا حرفيًا** يُطبع.
لا تخلط بينه وبين **سلاسل الهروب** (**sequenze di escape**) (مثل \n **للانتقال إلى سطر جديد**، \t للجدولة): فهذه تبدأ بالشرطة الخلفية \ وتمثل **حرفًا خاصًا** في النص، بينما %d يقول لـprintf كيف **يفسر ويعرض** قيمة – إنهما **آليتان مختلفتان**، حتى لو استُخدمتا معًا داخل علامتي الاقتباس نفسيهما.

تنبيه

انتبه إلى **الفراغات** وعلامات الترقيم داخل printf: **فاصلة منسية** أو فراغ خاطئ يفشل الترجمة. بيئة عملية لإجراء هذه التجارب الأولى دون تثبيت أي شيء هي onlinegdb.com: ليست أداة احترافية، لكنها مريحة للبدء، وتعمل من أي متصفح (حتى على الجهاز اللوحي أو الهاتف الذكي).



شكل 8: مخطط انسيابي: فعل واحد فقط، طباعة قيمة المتغير dui بواسطة %d.

7 تمرين عملي على onlinegdb.com

في هذا الدرس كتبنا ونفذنا معًا **برنامجًا** صغيرًا، انطلاقًا من **كود** **المثال** الموجود مسبقًا في **البيئة** **عبر** **الإنترنت**. الجزء الذي يهمنا هو دائمًا ما بين **القوسين** **المعقوفين** (**parentesi graffe**) للدالة `main` (أي كل ما بين قوس الفتح وقوس الإغلاق): هناك نحذف محتوى المثال و**نكتب** **كودنا**.

ملاحظة

كيف نكتب الأقواس المعقوفة على لوحة المفاتيح الإيطالية: قوس الفتح { نحصل عليه بـ `Alt Gr + Shift + è` (المفتاح الذي عليه القوس المربع المزدوج)؛ وقوس الإغلاق } بـ `Alt Gr + Shift +` المفتاح المجاور. إنها من أكثر الرموز استخدامًا في البرمجة: من المفيد **حفظ** هذه التركيبة **فورًا**.

تنبيه

خطأ شائع: **نسيان** **قوس** **الإغلاق** في نهاية البرنامج يسبب **خطأ** **ترجمة**. إذا لم يبدأ البرنامج ولم يكن السبب واضحًا، فأول ما يُفحص هو أن لكل **قوس** **مفتوح** **قوس** **إغلاق** يقابله.

تنبيه

مشكلة أكثر خفاء: إذا كان المتصفح قد فعّل **الترجمة** **التلقائية** للصفحة (مثل مترجم Google المدمج)، فقد تظهر بيئة البرمجة بنص مترجم بشكل خاطئ – فقد تظهر **الكلمة** **المفتاحية** `return` مترجمة و**لا** **يعود** **التعرف** **عليها** ككلمة في لغة C، مما يسبب أخطاء غريبة يصعب فهمها. إذا حدث ذلك، يجب **إيقاف** **الترجمة** **التلقائية** للصفحة من إعدادات المتصفح.

بعد كتابة الكود، فإن الزر **Run** (في الأعلى، باللون الأخضر) **ينفذ** **البرنامج**. إذا ظهر خطأ بدلًا من النتيجة المتوقعة، فالسبب غالبًا أحد ما يلي: **قوس** **منسي**، أو **فاصلة** أو **فاصلة** **منقوطة** **ناقصة**، أو **اسم** **متغير** مكتوب بطريقة مختلفة عما عُرّف به (تذكر: لغة C **case** **sensitive**).

8 ما يجب تذكره من هذا الدرس

- **العوامل** **الحسابية**: / * - + (الأخير يسمى **slash**).
- **عامل** **باقي** **القسمة** (%) يعطي **باقي** قسمة عددين صحيحين، وليس **نتيجة** **القسمة**.

- **عامل الإسناد** (=) ينسخ قيمة **اليمين** إلى متغير **اليسار** – و**ليس المساواة الرياضية**.
- قواعد أسماء المتغيرات: **أبجدي رقمي**، لا يمكن أن **يبدأ برقم**، بلا **رموز خاصة** (عدا **underscore**)، **case sensitive**.
- `printf("%d", variabile);` **يطبع على الشاشة** قيمة متغير باستخدام **محدد التنسيق** %d.
- على لوحة المفاتيح الإيطالية تُكتب **الأقواس المعقوفة** بـ`Alt Gr + Shift` + المفتاح المقابل – وتأتي دائمًا **زوجًا** (فتح/إغلاق).

مثال

للتحضير للدرس القادم: أنشئ متغيرين من نوع char، وأسند إليهما قيمتين تختارهما، واحسب **مجموعهما** و**باقي** قسمتهما (عامل الباقي)، ثم **اطبع** النتيجتين باستخدام **printf**.

9 مسرد: كلمات إيطالية تُستخدم في القسم

- **operatore / operando**: عامل / معامل
- **lavagna**: سبورة
- **resto**: باقي القسمة
- **assegnazione**: إسناد
- **punto e virgola**: فاصلة منقوطة (;)
- **parentesi graffe / tonde**: أقواس معقوفة { } / أقواس عادية ( )
- **stampare a video**: يطبع على الشاشة
- **diagramma di flusso**: مخطط انسيابي
- **errore di compilazione**: خطأ ترجمة
- **dichiarare**: يعرّف (متغيرًا)
- **doppi apici**: علامتا اقتباس مزدوجتان (")